

Explain Bitmask DP

Yes — this is a **Bitmask DP** solution, and the confusing part is that `dp[mask]` is **not storing true/false**. It stores:

What is the current sum of the bucket we are currently filling after using the elements represented by `mask`?

Let's build the idea slowly.

1. First understand the problem

We have:

```
C++
nums = [4, 3, 2, 3, 5, 2, 1]
k = 4
```

We need to divide all numbers into 4 subsets with equal sum.

```
Total sum = 4 + 3 + 2 + 3 + 5 + 2 + 1 = 20
target = 20 / 4 = 5
```

So ultimately we want something like:

```
Bucket 1 → [5]
Bucket 2 → [3, 2]
Bucket 3 → [3, 2]
Bucket 4 → [4, 1]
```

Every bucket must have:

```
sum = 5
```

2. Why Bitmask?

We need to remember:

```
Which elements have already been used?
```

For example:

```
nums = [4, 3, 2, 3]
index  0  1  2  3
```

We can represent whether each element is used using bits.

```
Bit = 1 → element is used
Bit = 0 → element is not used
```

For example:

```
mask = 0101
```

Reading from right to left:

```
index:  3  2  1  0
mask:   0  1  0  1
```

So:

```
nums[0] = 4 → used
nums[1] = 3 → not used
nums[2] = 2 → used
nums[3] = 3 → not used
```

Therefore:

```
0101 means {nums[0], nums[2]} are already used
```

3. What does `1 << i` mean?

This is the foundation of masking.

```
C++
```

```
1 << i
```

means:

Put 1 at position *i*.

Examples:

```
1 << 0 = 0001
1 << 1 = 0010
1 << 2 = 0100
1 << 3 = 1000
```

So:

```
C++
1 << i
```

creates a mask for the *i*-th element.

4. Checking whether an element is already used

This line:

```
C++
if (mask & (1 << i))
```

checks whether the *i*-th bit is 1.

Example:

```
mask = 0101
```

Let's check:

```
C++
i = 2
1 << 2 = 0100
```

Now:

```
0101
& 0100
-----
0100
```

Result is non-zero.

Therefore:

```
nums[2] is already used
```

So:

```
C++
if (mask & (1 << i)) {
    continue;
}
```

means:

```
If nums[i] is already used,
don't use it again.
```

5. How do we mark an element as used?

This line:

```
C++
int newMask = mask | (1 << i);
```

uses **OR**.

Suppose:

```
mask = 0101
```

and we want to use:

```
i = 1
```

Then:

```
1 << 1 = 0010
```

Now:

```
  0101
| 0010
-----
  0111
```

So:

```
C++
newMask = 0111;
```

Now elements:

```
index 0 → used
index 1 → used
index 2 → used
index 3 → not used
```

This is how we move from one DP state to another.

6. Now understand what `dp[mask]` means

The most important part:

```
C++
vector<int> dp(1 << n, -1);
```

There are:

```
2^n
```

possible subsets of `n` elements.

Each subset is represented by a mask.

So:

```
dp[mask]
```

means:

After using the elements represented by `mask`, what is the sum of the bucket that we are currently filling?

Example:

```
target = 5
```

Suppose:

```
dp[mask] = 3
```

That means:

```
All previous buckets are already complete.
Currently, we are filling a new bucket,
and its current sum is 3.
```

Visually:

```
Bucket 1 → 5 ✓
Bucket 2 → 5 ✓
Bucket 3 → 3 ← currently filling
```

7. Why initialize with `-1`?

```
C++
vector<int> dp(1 << n, -1);
```

Initially:

```
dp[mask] = -1
```

means:

This combination of used elements cannot be reached.

For example:

```
dp[0110] = -1
```

means:

```
We cannot reach a valid state using
exactly the elements represented by 0110.
```

Then:

```
C++
dp[0] = 0;
```

mask = 0 means:

```
0000
```

No elements are used.

Therefore:

```
Current bucket sum = 0
```

So:

```
C++
dp[0] = 0;
```

Starting state:

```
Used elements: none
Current bucket: []
Current sum: 0
```

8. Let's go through the code

```
C++
class Solution {
public:
    bool canPartitionKSubsets(vector<int>& nums, int k) {
```

Function starts.

Step 1: Number of elements

```
C++
int n = nums.size();
```

Example:

```
nums = [4, 3, 2, 3, 5, 2, 1]
n = 7
```

Step 2: Calculate total sum

```
C++
int sum = 0;
for (int x : nums) {
```

```
    sum += x;
}
```

Example:

```
sum = 20
```

Step 3: Check divisibility

```
C++
if (sum % k != 0) {
    return false;
}
```

If:

```
sum = 21
k = 4
```

Then:

```
21 % 4 != 0
```

We cannot divide the total sum equally.

So immediately:

```
C++
return false;
```

Step 4: Calculate target

```
C++
int target = sum / k;
```

Example:

```
sum = 20
k = 4

target = 5
```

Every subset must have:

```
sum = 5
```

9. DP initialization

```
C++
vector<int> dp(1 << n, -1);
```

If:

```
n = 3
```

Then:

```
1 << n = 1 << 3 = 8
```

So:

```
C++
dp = vector<int>(8, -1);
```

There are 8 possible masks:

```
000
001
010
011
100
101
110
111
```

Initially:

```
dp[000] = -1
dp[001] = -1
dp[010] = -1
...
```

Then:

```
C++
dp[0] = 0;
```

So:

```
mask = 000
dp[000] = 0
```

Meaning:

```
No elements used
Current bucket sum = 0
```

10. Loop through every possible mask

```
C++
for (int mask = 0; mask < (1 << n); mask++)
```

We visit every subset:

```
0000
0001
0010
0011
0100
...
1111
```

Each `mask` represents:

One particular set of elements that have already been used.

Skip unreachable states

```
C++
if (dp[mask] == -1) {
    continue;
}
```

For example:

```
dp[0110] = -1
```

Means:

```
We never found a valid way to reach this state.
```

So don't process it.

11. Try every unused element

```
C++
for (int i = 0; i < n; i++)
```

For every current state, try adding every number.

Check if already used

```
C++
```

```
if (mask & (1 << i)) {
    continue;
}
```

Example:

```
mask = 0101
```

Used:

```
nums[0]
nums[2]
```

If:

```
i = 2
```

Then:

```
mask      = 0101
1 << 2    = 0100
0101 & 0100 = 0100
```

Non-zero → already used.

So:

```
C++
continue;
```

12. Try adding the element

```
C++
int newSum = dp[mask] + nums[i];
```

Suppose:

```
target = 5
dp[mask] = 3
nums[i] = 2
```

Then:

```
newSum = 3 + 2 = 5
```

The current bucket is complete.

13. Don't exceed the target

```
C++
if (newSum > target) {
    continue;
}
```

Example:

```
Current bucket sum = 4
nums[i] = 3
target = 5
newSum = 7
```

We cannot put 3 into this bucket.

So:

```
C++
continue;
```

14. Create the new mask

```
C++
int newMask = mask | (1 << i);
```

This means:

Add `nums[i]` to the set of used elements.

Example:

```
mask = 0101
i = 1

1 << 1 = 0010
0101 | 0010 = 0111
```

So:

```
Old used elements:
0101 → index 0, 2

New used elements:
0111 → index 0, 1, 2
```

15. The most important line

```
C++
dp[newMask] = newSum % target;
```

This is the clever part.

Let's understand it carefully.

Suppose:

```
target = 5
```

Case 1: Bucket is not complete

```
Current sum = 2
Add = 1
newSum = 3
```

Then:

```
3 % 5 = 3
```

So:

```
C++
dp[newMask] = 3;
```

Meaning:

```
Current bucket has sum 3
```

Case 2: Bucket is complete

```
Current sum = 3
Add = 2
newSum = 5
```

Then:

```
5 % 5 = 0
```

So:

```
C++
dp[newMask] = 0;
```

This means:

```
One bucket is completed!
Start filling the next bucket from 0.
```

Visually:

Before:

Bucket 1 → [3, 2] = 5 
 Bucket 2 → currently starting
 dp[newMask] = 0

So % target automatically resets the current bucket.

16. Why don't we explicitly store the number of buckets?

This is a very important question.

We only store:

current bucket sum

We don't store:

number of completed buckets

Why?

Because the mask already tells us how many elements have been used.

And since:

Total sum of used elements

is fixed by the mask, if:

C++
 dp[mask] = 0

then the total sum of all used elements must be divisible by target.

Therefore, those used elements must form complete buckets.

Example:

target = 5
 Used elements total = 15
 15 = 5 + 5 + 5

So automatically:

3 buckets are complete

No need to store bucketCount.

17. Complete dry run

Let's use:

nums = [1, 2, 3, 4]
 k = 2

Total:

1 + 2 + 3 + 4 = 10

Therefore:

target = 5

We need:

[1, 4]
 [2, 3]

Initial state

mask = 0000

```
dp[0000] = 0
```

Meaning:

```
Used = {}
Current bucket sum = 0
```

Add nums[0] = 1

```
mask = 0000
1 << 0 = 0001
newMask = 0000 | 0001
          = 0001
```

New sum:

```
0 + 1 = 1
```

So:

```
dp[0001] = 1 % 5 = 1
```

State:

```
Used = {1}
Current bucket = [1]
Current sum = 1
```

From 0001, add nums[3] = 4

```
mask = 0001
i = 3
1 << 3 = 1000
```

New mask:

```
0001
|1000
-----
1001
```

New sum:

```
dp[0001] + 4
= 1 + 4
= 5
```

Then:

```
C++
dp[1001] = 5 % 5 = 0;
```

Meaning:

```
Bucket complete!
[1, 4] = 5
Current bucket sum = 0
```

From 1001, add nums[1] = 2

```
mask = 1001
1 << 1 = 0010
1001
|0010
-----
1011
```

New sum:

```
0 + 2 = 2
```

So:

```
C++
dp[1011] = 2;
```

Current state:

```
Bucket 1 = [1, 4] = 5 ✓
Bucket 2 = [2]
Current sum = 2
```

Add nums[2] = 3

```
mask = 1011
1 << 2 = 0100
1011
|0100
-----
1111
```

New sum:

```
2 + 3 = 5
```

Then:

```
C++
dp[1111] = 5 % 5 = 0;
```

Now:

```
Bucket 1 = [1, 4] = 5
Bucket 2 = [2, 3] = 5
```

All elements used:

```
mask = 1111
```

And:

```
dp[1111] = 0
```

Therefore:

```
true
```

18. What does the final mask mean?

```
C++
(1 << n) - 1
```

This creates:

```
111111...111
```

with n bits.

Example:

```
n = 4
1 << 4 = 10000
10000 - 1 = 01111
```

Ignoring the leading zero:

```
1111
```

So:

```
C++
(1 << n) - 1
```

means:

Every element has been used.

Therefore:

```
C++
dp[(1 << n) - 1]
```

represents the state where all numbers are used.

19. There is a mistake in your code

Your final line is:

```
C++
return dp[(1 << n) - 1] = 0;
```

This uses:

```
=
```

which is **assignment**, not comparison.

You are assigning 0.

It should be:

```
C++
return dp[(1 << n) - 1] == 0;
```

Meaning:

After using all elements, is the current bucket sum 0?

If yes:

```
All buckets were completed exactly.
```

20. The complete flow

The whole DP can be visualized like this:



Eventually:

```
mask = 1111111
```

↓

```
dp[allUsedMask] == 0 ?
```

```
YES → true
```

```
NO → false
```

The main intuition to remember

```
mask
↓
Tells which numbers are already used.

dp[mask]
↓
Tells the sum of the current incomplete bucket.

newMask = mask | (1 << i)
↓
Use nums[i].

dp[newMask] = (dp[mask] + nums[i]) % target
↓
If target is reached → bucket resets to 0.
Otherwise → continue filling the same bucket.
```

The corrected final code is:

```
C++
class Solution
{
public:
    bool canPartitionKSubsets(vector<int>& nums, int k)
    {
        int n = nums.size();

        // Calculate total sum
        int sum = 0;

        for(int x : nums)
        {
            sum += x;
        }

        // Equal partition is impossible
        if(sum % k != 0)
        {
            return false;
        }

        // Every bucket must have this sum
        int target = sum / k;

        // dp[mask] = current sum of the bucket
        // -1 means this state cannot be reached
        vector<int> dp(1 << n, -1);

        // No elements used
        // Current bucket sum = 0
        dp[0] = 0;

        // Try every subset
        for(int mask = 0; mask < (1 << n); mask++)
        {
            // Skip unreachable states
            if(dp[mask] == -1)
            {
                continue;
            }

            // Try adding every unused number
            for(int i = 0; i < n; i++)
            {
                // nums[i] is already used
                if(mask & (1 << i))
                {
                    continue;
                }

                // Add nums[i] to current bucket
                int newSum = dp[mask] + nums[i];

                // Bucket cannot exceed target
                if(newSum > target)
                {
                    continue;
                }

                // Mark nums[i] as used
                int newMask = mask | (1 << i);

                // If bucket becomes target,
                // modulo makes it 0 and starts next bucket
                dp[newMask] = newSum % target;
            }
        }
    }
};
```

```

    }
    // All elements must be used
    // and the final bucket must also be complete
    return dp[(1 << n) - 1] == 0;
}
};

```

The **single most important concept** here is:

```

C++
dp[mask] = sum of the current incomplete bucket

```

and:

```

C++
dp[newMask] = (dp[mask] + nums[i]) % target;

```

That `% target` is what allows this DP to simulate **completing one bucket and automatically starting the next bucket** without explicitly storing all `k` buckets.

Absolutely. Let's do a **complete dry run**, slowly, and track:

1. `mask`
2. Which elements are used
3. `dp[mask]`
4. Which `nums[i]` we try
5. `1 << i`
6. `mask & (1 << i)`
7. `newMask`
8. `newSum`
9. How `% target` completes a bucket

Example

Let's take a very small input:

```

C++
nums = [1, 2, 3, 4]
k = 2

```

We need to divide the array into 2 subsets having equal sum.

Step 1: Calculate total sum

```

1 + 2 + 3 + 4 = 10

```

Since:

```

10 % 2 = 0

```

It is possible to divide equally.

So:

```

target = 10 / 2 = 5

```

Each bucket must have sum:

```

5

```

A possible answer is:

```

Bucket 1 = [1, 4] = 5
Bucket 2 = [2, 3] = 5

```

Step 2: Understand the indexes

```
nums = [1, 2, 3, 4]

index  0  1  2  3
nums   1  2  3  4
```

Since $n = 4$, we have:

```
2^4 = 16 masks
```

Masks go from:

```
0000
0001
0010
0011
0100
0101
0110
0111
1000
1001
1010
1011
1100
1101
1110
1111
```

We will read the bits like this:

```
mask bits:  index 3  index 2  index 1  index 0
```

For example:

```
mask = 0101

index:  3 2 1 0
bits:   0 1 0 1
```

Therefore:

```
nums[0] = 1 → used
nums[1] = 2 → not used
nums[2] = 3 → used
nums[3] = 4 → not used
```

So:

```
0101 = {1, 3}
```

Step 3: Initialize DP

```
C++
vector<int> dp(1 << n, -1);
```

Since:

```
1 << 4 = 16
```

We create:

```
dp[0 ... 15]
```

Initially:

```
dp[0000] = -1
dp[0001] = -1
dp[0010] = -1
...
dp[1111] = -1
```

Then:

```
C++
dp[0] = 0;
```

So:

```
dp[0000] = 0
```

Meaning:

```
Used elements = {}
Current bucket sum = 0
```

Initial state:

```
mask = 0000
dp[0000] = 0
```

FULL DRY RUN

The loop starts:

```
C++
for(int mask = 0; mask < (1 << n); mask++)
```

So first:

MASK = 0000

```
mask = 0000
dp[0000] = 0
```

This state is reachable.

Current situation:

```
Used elements = {}
Current bucket = []
Current bucket sum = 0
```

Now try every i .

Try $i = 0$

We want to check whether `nums[0] = 1` is already used.

```
C++
mask & (1 << i)
```

First:

```
1 << 0 = 0001
```

Now:

```
mask      = 0000
1 << 0    = 0001
-----
0000 & 0001 = 0000
```

Result is 0.

Therefore:

```
nums[0] is NOT used
```

Now calculate:

```
C++
newSum = dp[mask] + nums[i];
```

So:

```
newSum = 0 + 1
        = 1
```

Check:

```
1 > 5 ?
```

No.

So we can use it.

Now create the new mask:

```
C++
newMask = mask | (1 << i);
```

```
mask      = 0000
1 << 0    = 0001
-----
newMask    = 0001
```

Now:

```
C++
dp[newMask] = newSum % target;
```

So:

```
dp[0001] = 1 % 5
          = 1
```

Our new state is:

```
mask = 0001
Used elements = {1}
Current bucket = [1]
Current bucket sum = 1
```

DP now contains:

```
dp[0000] = 0
dp[0001] = 1
```

Back to MASK = 0000

Now try:

```
i = 1
nums[1] = 2
```

Check:

```
1 << 1 = 0010
```

Now:

```
0000 & 0010 = 0000
```

So index 1 is not used.

Calculate:

```
newSum = dp[0000] + nums[1]
        = 0 + 2
        = 2
```

Valid because:

```
2 <= 5
```

Create new mask:

```
0000 | 0010 = 0010
```

Store:

```
dp[0010] = 2 % 5
          = 2
```

State:

```
mask = 0010
Used elements = {2}
```

```
Current bucket = [2]
Current bucket sum = 2
```

Back to MASK = 0000

Try:

```
i = 2
nums[2] = 3
```

Create the bit:

```
1 << 2 = 0100
```

Check:

```
0000 & 0100 = 0000
```

Not used.

Calculate:

```
newSum = 0 + 3
        = 3
```

Valid.

New mask:

```
0000 | 0100 = 0100
```

Store:

```
dp[0100] = 3 % 5
          = 3
```

State:

```
Used elements = {3}
Current bucket = [3]
Current bucket sum = 3
```

Back to MASK = 0000

Try:

```
i = 3
nums[3] = 4
```

Create bit:

```
1 << 3 = 1000
```

Check:

```
0000 & 1000 = 0000
```

Not used.

Calculate:

```
newSum = 0 + 4
        = 4
```

Valid.

New mask:

```
0000 | 1000 = 1000
```

Store:

```
dp[1000] = 4 % 5
          = 4
```

After processing `mask = 0000`, DP looks like:

Mask	Used elements	dp[mask]
0000	{}	0
0001	{1}	1
0010	{2}	2
0100	{3}	3
1000	{4}	4

MASK = 0001

Now:

```
mask = 0001
dp[0001] = 1
```

Meaning:

```
nums[0] = 1 is already used
Current bucket = [1]
Current sum = 1
```

Try i = 0

Check:

```
mask      = 0001
1 << 0    = 0001
```

AND:

```
0001 & 0001 = 0001
```

Non-zero.

Therefore:

```
nums[0] is already used
```

So:

```
C++
continue;
```

We cannot use 1 again.

Try i = 1

```
nums[1] = 2
```

Create bit:

```
1 << 1 = 0010
```

Check:

```
0001 & 0010 = 0000
```

Not used.

Calculate:

```
newSum = dp[0001] + nums[1]
        = 1 + 2
        = 3
```

Valid.

New mask:

```
0001 | 0010
=
0011
```

Store:

```
dp[0011] = 3 % 5
= 3
```

Meaning:

```
Used = {1, 2}
Current bucket = [1, 2]
Current sum = 3
```

Try i = 2

```
nums[2] = 3
```

Check:

```
1 << 2 = 0100
```

```
0001 & 0100 = 0000
```

Not used.

Calculate:

```
newSum = 1 + 3
= 4
```

Valid.

New mask:

```
0001 | 0100
=
0101
```

Store:

```
dp[0101] = 4
```

State:

```
Used = {1, 3}
Current bucket = [1, 3]
Current sum = 4
```

Try i = 3

```
nums[3] = 4
```

Check:

```
1 << 3 = 1000
```

```
0001 & 1000 = 0000
```

Not used.

Calculate:

```
newSum = 1 + 4
= 5
```

Exactly equal to target!

Create new mask:

```
0001 | 1000
=
1001
```

Now the important line:

```
C++
dp[newMask] = newSum % target;
```

So:

```
dp[1001] = 5 % 5
= 0
```

Why 0?

Because:

```
[1, 4] = 5
```

One bucket is complete.

So now:

```
Bucket 1 = [1, 4] = 5 ✓
Start next bucket:
Current bucket sum = 0
```

Therefore:

```
mask = 1001
dp[1001] = 0
```

This is the key transition.

MASK = 0010

Now:

```
dp[0010] = 2
```

Meaning:

```
Used = {2}
Current bucket = [2]
Current sum = 2
```

Let's try adding 3.

Index:

```
i = 2
nums[2] = 3
```

Check:

```
0010 & 0100 = 0000
```

Not used.

Calculate:

```
newSum = 2 + 3
= 5
```

New mask:

```
0010 | 0100
=
0110
```

Store:

```
dp[0110] = 5 % 5
          = 0
```

Meaning:

```
Bucket complete:
[2, 3] = 5 ✓
```

Now:

```
dp[0110] = 0
```

Important: Multiple paths can reach different masks

At this point, some masks have been reached:

```
0000 → 0
0001 → 1
0010 → 2
0100 → 3
1000 → 4
0011 → 3
0101 → 4
0110 → 0
1001 → 0
...
```

We don't need to manually explore every one of the 16 masks in the same detail.

The important successful path we want is:

```
0000
↓ add 1
0001
↓ add 4
1001
↓ add 2
1011
↓ add 3
1111
```

Now let's follow that exact path completely.

Continue from MASK = 1001

Recall:

```
mask = 1001
dp[1001] = 0
```

Bits:

```
index:  3 2 1 0
mask:   1 0 0 1
```

Used:

```
nums[0] = 1
nums[3] = 4
```

Already completed:

```
Bucket 1 = [1, 4] = 5
```

Now:

```
Current bucket sum = 0
```

Remaining numbers:

```
2, 3
```

Try i = 0

```
1001 & 0001 = 0001
```

Already used.

Skip.

Try i = 1

```
nums[1] = 2
```

Bit:

```
1 << 1 = 0010
```

Check:

```
1001 & 0010 = 0000
```

Not used.

Calculate:

```
newSum = dp[1001] + nums[1]
        = 0 + 2
        = 2
```

Valid.

New mask:

```
1001 | 0010
=
1011
```

Store:

```
dp[1011] = 2 % 5
          = 2
```

State:

```
Bucket 1 = [1, 4] = 5 ✓
Current bucket = [2]
Current sum = 2
```

MASK = 1011

Now:

```
mask = 1011
dp[1011] = 2
```

Bits:

```
index:  3 2 1 0
mask:    1 0 1 1
```

Used:

```
nums[0] = 1
nums[1] = 2
nums[3] = 4
```

Unused:

```
nums[2] = 3
```

Current state:

```
Bucket 1 = [1, 4] = 5 ✓
Current bucket = [2]
Current sum = 2
```

Try i = 2

```
nums[2] = 3
```

Bit:

```
1 << 2 = 0100
```

Check:

```
1011 & 0100
=
0000
```

Not used.

Calculate:

```
newSum = dp[1011] + nums[2]
        = 2 + 3
        = 5
```

Exactly the target.

Create new mask:

```
1011 | 0100
=
1111
```

Now:

```
C++
dp[1111] = 5 % 5;
```

Therefore:

```
dp[1111] = 0
```

Meaning:

```
Bucket 2 is complete.
```

Final state:

```
Bucket 1 = [1, 4] = 5 ✓
Bucket 2 = [2, 3] = 5 ✓
```

All elements are used:

```
mask = 1111
```

And:

```
dp[1111] = 0
```

Therefore:

```
C++
return dp[(1 << n) - 1] == 0;
```

Since:

```
(1 << 4) - 1
=
10000 - 1
=
1111
```

So:

```
C++
return dp[1111] == 0;
```

Which is:

```
return 0 == 0
```

Therefore:


```
true
```

Complete successful path visually

START

```
mask = 0000
dp = 0

No elements used
Current bucket = []
```

Add 1

```
0000
|
| add nums[0] = 1
v
0001

dp[0001] = 1
Current bucket = [1]
```

Add 4

```
0001
|
| add nums[3] = 4
v
1001

newSum = 1 + 4 = 5
dp[1001] = 5 % 5 = 0
```

So:

```
Bucket [1, 4] complete ✓
Current bucket resets to []
```

Add 2

```
1001
|
| add nums[1] = 2
v
1011

dp[1011] = 2
Current bucket = [2]
```

Add 3

```
1011
|
| add nums[2] = 3
v
1111

newSum = 2 + 3 = 5
dp[1111] = 5 % 5 = 0
```

So:

```
Bucket [2, 3] complete ✓
```

Final:

```
mask = 1111
      ↑↑↑↑
      all elements used

dp[1111] = 0
```

Therefore:

```
true
```

The masking operations in one table

Operation	Example	Meaning
<code>1 << i</code>	<code>1 << 2 = 0100</code>	Create a bit for index 2
<code>mask & (1 << i)</code>	<code>0101 & 0100 = 0100</code>	Check if index 2 is used
<code>mask (1 << i)</code>	<code>0101 0010 = 0111</code>	Mark index 1 as used
<code>dp[mask]</code>	<code>dp[1011] = 2</code>	Current incomplete bucket has sum 2
<code>newSum % target</code>	<code>5 % 5 = 0</code>	Bucket completed, reset sum

The entire idea in one sentence

mask tells us which elements are used,
and dp[mask] tells us how much of the current bucket has been filled.

Then every transition is simply:

```
C++
Take an unused element
↓
Check if it fits in current bucket
↓
Mark it as used
↓
If bucket reaches target → dp becomes 0
↓
Otherwise → keep the current bucket sum
```

So the DP is basically exploring **all possible subsets of used elements**, while the value stored in each mask tells us **where we currently are inside the process of filling buckets**.

ChatGPT can make mistakes. Check important info.